

## Лабораторная работа №4

### Разработка транслятора заданных конструкций языка СИ++. Разработка синтаксического анализатора, обнаруживающего максимальное число ошибок.

#### Теоретическая часть

Любая программа потенциально содержит множество ошибок самого разного уровня. Например, ошибки могут быть:

- Лексическими, такими как неверно записанные идентификаторы, ключевые слова или операторы;
- Синтаксическими, например арифметические выражения с несбалансированными скобками;
- Семантическими, такими как операторы, применяемые к несовместимым с ними операндам;
- Логическими, например бесконечная рекурсия

Зачастую основные действия по выявлению ошибок и восстановлению после них концентрируются в фазе синтаксического анализа. Одна из причин этого состоит в том, что многие ошибки по природе своей являются синтаксическими или проявляются, когда поток токенов, идущий от лексического анализатора, нарушает определяющие язык программирования грамматические правила.

Обработчик ошибок синтаксического анализатора имеет очень просто формулируемые цели:

- Он должен ясно и точно сообщать о наличии ошибок;
- Он должен обеспечивать быстрое восстановление после ошибки, чтобы продолжить поиск следующих ошибок;
- Он не должен существенно замедлять обработку корректной программы.

Эффективная реализация этих целей представляет собой весьма сложную задачу.

К счастью, обычные ошибки достаточно просты, и для их обработки часто достаточно относительно простых механизмов обработки ошибок. В некоторых случаях, однако, ошибка может произойти задолго до момента её выявления (и за много строк кода до места её выявления), и определить её природу весьма непросто. В некоторых ситуациях обработчик ошибок, по сути, должен попросту догадаться, что именно имел в виду программист, когда писал программу.

Как именно обработчик ошибок должен сообщить о наличии ошибки? Как минимум – сообщить о месте в исходной программе, где была выявлена ошибка, т.к. на самом деле ошибка находится, как правило, в пределах нескольких предыдущих токенов. Общая стратегия, используемая многими компиляторами, заключается в выводе ошибочной строки с указанием позиции, в которой обнаружена ошибка. Если имеется обоснованное предположение о природе ошибки, вывод включает диагностическое пояснение типа «отсутствие точки с запятой в данной позиции».

После того, как ошибка выявлена, в чём же должно состоять восстановление синтаксического анализатора? В большинстве случаев окончание анализа после выявления первой ошибки не является хорошим решением, т.к. восстановление и продолжение анализа могло бы выявить ряд последующих ошибок. Обычно восстановление после ошибки заключается в том, что синтаксический анализатор пытается восстановить некоторое своё состояние, в котором продолжается обработка входного потока так, как если бы он был корректным.

Неадекватное восстановление после ошибки может привести к настоящей лавине ложных ошибок, которые не были допущены программистом, а появились вследствие изменений состояния синтаксического анализатора в процессе восстановления после ошибки. Аналогично восстановление после синтаксической ошибки может привести к ложным семантическим ошибкам, которые выявятся позже, на стадии семантического анализа или генерации кода. Например, при восстановлении после ошибки синтаксический анализатор может пропустить объявление некоторой переменной.

Консервативная стратегия компилятора состоит в запрете сообщений об ошибках, которые возникают во входном потоке слишком близко. После обнаружения одной синтаксической ошибки компилятор должен успешно проанализировать несколько входных токенов и только после этого разрешить вывод других сообщений об ошибках.

### **Стратегии восстановления после ошибок**

Существуют различные стратегии, которые синтаксический анализатор может использовать для восстановления после синтаксической ошибки. Вот некоторые из них:

*Режим паники.* Эта стратегия реализуется проще всего и может использоваться большинством методов синтаксического анализа. При обнаружении ошибки синтаксический анализатор пропускает входные символы по одному, пока не будет найден один из специально определённого множества синхронизирующих токенов. Обычно такими токенами являются разделители, например `end` или точка с запятой, роль которых в исходной программе совершенно очевидна. Разработчик компилятора, естественно, должен выбрать синхронизирующие токены исходя из особенностей исходного языка. Хотя такая коррекция часто пропускает значительное количество символов без проверки на наличие ошибок, она достаточно проста и, в отличие от некоторых методов, гарантирует отсутствие заикливания. В случаях, когда несколько ошибок в одной инструкции встречаются очень редко, этот метод работает вполне адекватно.

*Уровень фразы.* При обнаружении ошибки синтаксический анализатор может выполнить локальную коррекцию оставшегося входного потока. Т.о., он может заменить префикс остальной части потока некоторой строкой, которая позволит синтаксическому анализатору продолжить работу. Типичная локальная коррекция может состоять в удалении лишней точки с запятой или вставке недостающей. Выбор способа локальной коррекции – прерогатива разработчика компилятора. Естественно, нужно быть предельно осторожным при выборе способа коррекции, чтобы он не привёл, например, к бесконечному циклу.

Этот тип замещения может скорректировать любую входную строку и используется в ряде компиляторов, исправляющих ошибки. Основной его недостаток заключается в сложности обработки ситуации, когда реально ошибка располагается до точки обнаружения.

*Продукции ошибок.* Если известны наиболее распространённые ошибки, можно расширить грамматику языка productions, порождающими ошибочные конструкции. Затем строится синтаксический анализатор на основе такой расширенной грамматики. Если синтаксический анализатор использует одну из «ошибочных» productions, можно сгенерировать корректное диагностическое сообщение для распознанной синтаксическим анализатором ошибки.

## Нерекурсивный предиктивный анализ

Является одним из вариантов синтаксического анализа. Может быть построен с помощью явного использования стека. Ключевая проблема предиктивного анализа заключается в определении продукции, которую следует применить к нетерминалу. Нерекурсивный синтаксический анализатор, представленный на рис. 1, ищет необходимую для анализа продукцию в таблице разбора (далее будет показано, как строится эта таблица на основе грамматики).

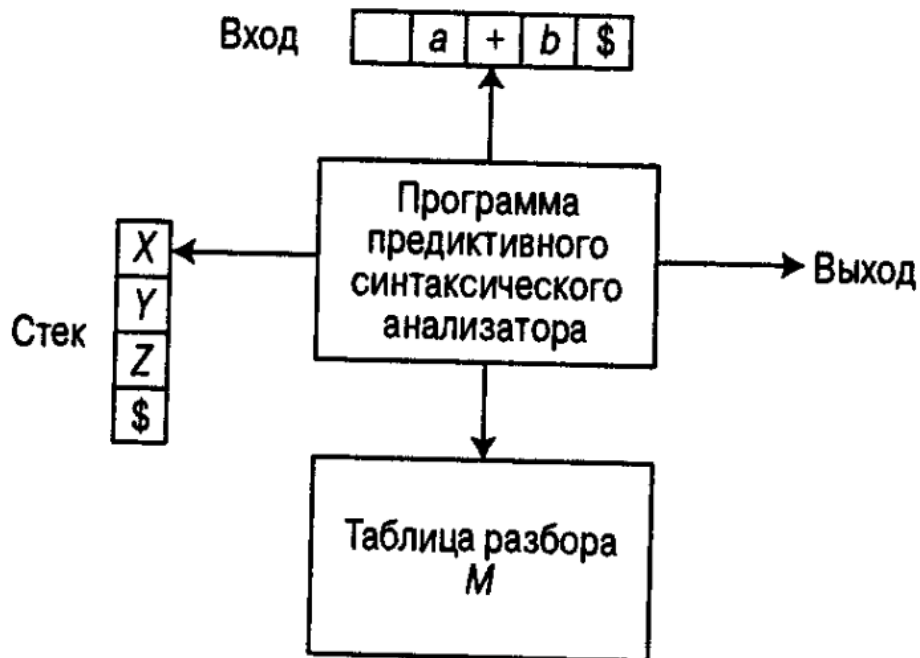


Рис 1. Модель нерекурсивного предиктивного синтаксического анализатора

Для изложения дальнейшего материала введём некоторые простейшие соглашения (они действуют по умолчанию, если не сказано иного):

1. Прописными буквами латинского алфавита будут обозначаться нетерминалы;
2. Строчными буквами латинского алфавита – терминалы;
3. Символ \$ обозначает дно стека, конец строки и т.п.
4. Символ  $\epsilon$  обозначает пустой символ.

Предиктивный синтаксический анализатор, управляемый таблицей, имеет входной буфер, стек, таблицу разбора и выходной поток. Входной буфер содержит анализируемую строку с маркером её правого конца – специальным символом. Стек содержит последовательность символов грамматики с \$ на конце. Изначально стек содержит стартовый символ грамматики непосредственно над символом \$. Таблица разбора представляет собой двухмерный массив  $M[A, a]$ , где  $A$  – нетерминал, а  $a$  – терминал или символ \$.

Синтаксический анализатор управляется программой, которая работает следующим образом. Программа рассматривает  $X$  – символ на вершине стека, и  $a$  – текущий входной символ. Эти 2 символа определяют действия синтаксического анализатора. Имеется 3 варианта:

- Если  $X = a = \$$ , синтаксический анализатор прекращает работу и сообщает об успешном завершении разбора;
- Если  $X = a \neq \$$ , синтаксический анализатор снимает со стека  $X$  и перемещает указатель входного потока к следующему символу;

- Если  $X$  представляет собой нетерминал, программа рассматривает запись  $M[X,a]$  из таблицы разбора  $M$ . Эта запись представляет собой либо  $X$ -продукцию грамматики, либо запись об ошибке. Если, например,  $M[X,a] = \{X \rightarrow UVW\}$ , синтаксический анализатор замещает  $X$  на вершине стека на  $WVU$  (с  $U$  на вершине стека). В данном случае полагается, что в качестве выхода синтаксический анализатор просто выводит использованную продукцию, но, конечно же, здесь может выполняться любой необходимый код. Если  $M[X,a] = \text{error}$ , синтаксический анализатор вызывает программу восстановления после ошибки.

**Алгоритм:** нерекурсивный предиктивный анализ

**Вход:** строка  $w$  и таблица разбора  $M$  грамматики  $G$ .

**Выход:** в случае  $w \in L(G)$  – левое порождение  $w$ ; в противном случае – сообщение об ошибке.

**Метод:** Изначально синтаксический анализатор находится в конфигурации с  $\$S$  (на его вершине – стартовый символ  $S$  грамматики  $G$ ) и строкой  $w\$$  во входном буфере. Программа, использующая для анализа входного потока таблицу предиктивного разбора  $M$ , приведена ниже:

Установить указатель  $ip$  на первый символ  $w\$$ ;

**repeat**

    обозначим через  $X$  символ на вершине стека, а через  $a$  – символ, на который указывает  $ip$ .

**if**  $X$  – терминал или  $\$$  **then**

**if**  $X = a$  **then**

            снять со стека  $X$  и переместить  $ip$

**else** error()

**else** /\* $X$  - нетерминал\*/

**if**  $M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$  **then begin**

            снять  $X$  со стека;

            Поместить в стек  $Y_1, Y_2, \dots, Y_k$  с  $Y_1$  на вершине стека;

            Вывести продукцию  $X \rightarrow Y_1 Y_2 \dots Y_k$

**end**

**else** error()

**until**  $X = \$$  /\* Стек пуст \*/

**Пример 1.** Рассмотрим грамматику со следующими productions

$E \rightarrow TA$

$A \rightarrow +TA \mid \$$

$T \rightarrow FB$

$B \rightarrow *FB \mid \$$

$F \rightarrow (E) \mid \text{id}$

Таблица предиктивного анализа этой грамматики показана на рис 2. Пустые ячейки означают ошибки; непустые указывают продукции, с помощью которых заменяются нетерминалы на вершине стека.

| Нетерминал | Входной символ     |                             |                     |                     |                             |                             |
|------------|--------------------|-----------------------------|---------------------|---------------------|-----------------------------|-----------------------------|
|            | id                 | +                           | *                   | (                   | )                           | \$                          |
| E          | $E \rightarrow TA$ |                             |                     | $E \rightarrow TA$  |                             |                             |
| A          |                    | $A \rightarrow +TA$         |                     |                     | $A \rightarrow \varepsilon$ | $A \rightarrow \varepsilon$ |
| T          | $T \rightarrow FB$ |                             |                     | $T \rightarrow FB$  |                             |                             |
| B          |                    | $B \rightarrow \varepsilon$ | $B \rightarrow *FB$ |                     | $B \rightarrow \varepsilon$ | $B \rightarrow \varepsilon$ |
| F          | $F \rightarrow id$ |                             |                     | $F \rightarrow (E)$ |                             |                             |

Рис 2. Таблица разбора

При входном потоке **id+id\*id** предиктивный синтаксический анализатор осуществляет последовательность перемещений, показанную на рис. 3. Входной указатель при перемещении указывает на крайний слева символ в столбце «Вход».

| Стек   | Вход               | Выход                       |
|--------|--------------------|-----------------------------|
| \$E    | <b>id+id*id</b> \$ |                             |
| \$AT   | <b>id+id*id</b> \$ | $E \rightarrow TA$          |
| \$ABF  | <b>id+id*id</b> \$ | $T \rightarrow FB$          |
| \$ABid | <b>id+id*id</b> \$ | $F \rightarrow id$          |
| \$AB   | <b>+id*id</b> \$   |                             |
| \$A    | <b>+id*id</b> \$   | $B \rightarrow \varepsilon$ |
| \$AT+  | <b>+id*id</b> \$   | $A \rightarrow +TA$         |
| \$AT   | <b>id*id</b> \$    |                             |
| \$ABF  | <b>id*id</b> \$    | $T \rightarrow FB$          |
| \$ABid | <b>id*id</b> \$    | $F \rightarrow id$          |
| \$AB   | <b>*id</b> \$      |                             |
| \$ABF* | <b>*id</b> \$      | $B \rightarrow *FB$         |
| \$ABF  | <b>id</b> \$       |                             |
| \$ABid | <b>id</b> \$       | $F \rightarrow id$          |
| \$AB   | <b>\$</b>          |                             |
| \$A    | <b>\$</b>          | $B \rightarrow \varepsilon$ |
| \$     | <b>\$</b>          | $A \rightarrow \varepsilon$ |

Рис 3. Перемещения предиктивного синтаксического анализатора

### FIRST и FOLLOW

При построении предиктивного синтаксического анализатора весьма полезными оказываются две функции, связанные с грамматикой  $G$ , - FIRST и FOLLOW, которые обеспечивают заполнение таблицы предиктивного анализа грамматики  $G$ . Множества токенов, порождаемые функцией FOLLOW, могут также использоваться как синхронизирующие токены в процессе восстановления после ошибки в «режиме паники».

Если  $\alpha$  – произвольная строка символов грамматики, то определим  $FIRST(\alpha)$  как множество терминалов, с которых начинаются строки, выводимые из  $\alpha$ . Если  $\alpha \Rightarrow \varepsilon$ , то  $\varepsilon \in FIRST(\alpha)$

Определим  $FOLLOW(A)$  для нетерминала  $A$  как множество терминалов  $a$ , которые могут располагаться непосредственно справа от  $A$  в некоторой сентенциальной форме, т.е. множество терминалов  $a$ , таких, что существует порождение вида  $S \Rightarrow \alpha A a \beta$  для некоторых  $\alpha$  и  $\beta$ . В процессе приведения между  $A$  и  $a$

могут появиться символы, но они порождают  $\varepsilon$  и в конечном счёте исчезают. Если  $A$  может оказаться крайним справа символом некоторой сентенциальной формы, то  $\$ \in \text{FOLLOW}(A)$ .

Чтобы вычислить  $\text{FIRST}(X)$  для всех символов грамматики  $X$ , применяются следующие правила до тех пор, пока ни к одному из множеств  $\text{FIRST}$  не смогут быть добавлены ни терминалы, ни  $\varepsilon$ .

- Если  $X$  – терминал, то  $\text{FIRST}(X) = \{X\}$
- Если имеется продукция  $X \rightarrow \varepsilon$ , добавим  $\varepsilon$  к  $\text{FIRST}(X)$
- Если  $X$  – нетерминал и имеется продукция  $X \rightarrow Y_1 Y_2 \dots Y_k$ , то поместим  $a$  в  $\text{FIRST}(X)$ , если для некоторого  $i$   $a \in \text{FIRST}(Y_i)$  и  $\varepsilon$  входит во все множества  $\text{FIRST}(Y_1), \dots, \text{FIRST}(Y_{i-1})$ , т.е.  $Y_1 \dots Y_{i-1} \Rightarrow \varepsilon$ . Если  $\varepsilon$  имеется во всех  $\text{FIRST}(Y_i)$ ,  $i=1..k$ , то добавляем  $\varepsilon$  к  $\text{FIRST}(X)$ . Например, всё, что находится в множестве  $\text{FIRST}(Y_1)$ , есть и в множестве  $\text{FIRST}(X)$ . Если  $Y_1$  не порождает  $\varepsilon$ , то больше ничего не добавляется к  $\text{FIRST}(X)$ , но если  $Y_1 \Rightarrow \varepsilon$ , то к  $\text{FIRST}(X)$  добавляется  $\text{FIRST}(Y_2)$  и т.д.

Теперь для любой строки  $X_1 X_2 \dots X_n$  вычислить  $\text{FIRST}$  можно следующим образом: добавим к  $\text{FIRST}(X_1 \dots X_n)$  все не- $\varepsilon$  символы из  $\text{FIRST}(X_1)$ . Добавим также все не- $\varepsilon$  символы из  $\text{FIRST}(X_2)$ , если  $\varepsilon \in \text{FIRST}(X_1)$ , все не- $\varepsilon$  символы из  $\text{FIRST}(X_3)$ , если  $\varepsilon$  имеется как в  $\text{FIRST}(X_1)$ , так и в  $\text{FIRST}(X_2)$  и т.д. И, наконец, добавим  $\varepsilon$  к  $\text{FIRST}(X_1 X_2 \dots X_n)$  если для всех  $i$   $\text{FIRST}(X_i)$  содержит  $\varepsilon$ .

Чтобы вычислить  $\text{FOLLOW}(A)$  для всех нетерминалов  $A$ , будем применять следующие правила до тех пор, пока ни к одному множеству  $\text{FOLLOW}$  нельзя будет добавить ни одного символа.

- Поместим  $\$$  в  $\text{FOLLOW}(S)$ , где  $S$  – стартовый символ, а  $\$$  – правый ограничитель входного потока.
- Если имеется продукция  $A \rightarrow \alpha B \beta$ , то все элементы множества  $\text{FIRST}(\beta)$ , кроме  $\varepsilon$ , помещаются в множество  $\text{FOLLOW}(B)$ .
- Если имеется продукция  $A \rightarrow \alpha B$  или  $A \rightarrow \alpha B \beta$ , где  $\text{FIRST}(\beta)$  содержит  $\varepsilon$  (т.е.  $\beta \Rightarrow \varepsilon$ ), то все элементы из множества  $\text{FOLLOW}(A)$  помещаются в множество  $\text{FOLLOW}(B)$ .

**Пример 2.** Обратимся вновь к грамматике из примера 1

Для неё

$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$

$\text{FIRST}(A) = \{ +, \varepsilon \}$

$\text{FIRST}(B) = \{ *, \varepsilon \}$

$\text{FOLLOW}(E) = \text{FOLLOW}(A) = \{ ), \$ \}$

$\text{FOLLOW}(T) = \text{FOLLOW}(B) = \{ +, ), \$ \}$

$\text{FOLLOW}(F) = \{ +, *, ), \$ \}$

### Построение таблиц предиктивного анализа

Идея, лежащая в основе алгоритма построения таблицы, состоит в следующем. Предположим, что  $A \rightarrow \alpha$  представляет собой продукцию, у которой  $a \in \text{FIRST}(\alpha)$ . Тогда синтаксический анализатор заменит  $A$  строкой  $\alpha$  при текущем входном символе  $a$ . Единственная сложность возникает при  $\alpha = \varepsilon$  или  $\alpha \Rightarrow \varepsilon$ . В этом случае мы снова должны заменить  $A$  на  $\alpha$ , если текущий входной символ имеется в  $\text{FOLLOW}(A)$  или из входного потока получен  $\$$ , который входит в  $\text{FOLLOW}(A)$ .

### **Алгоритм**

**Вход:** грамматика  $G$

**Выход:** таблица анализа  $M$ .

**Метод:**

1. Для каждой продукции грамматики  $A \rightarrow \alpha$  выполнять шаги 2 и 3
2. Для каждого терминала  $a$  из  $FIRST(\alpha)$  добавляем  $A \rightarrow \alpha$  в ячейку  $M[A, a]$
3. Если в  $FIRST(\alpha)$  входит  $\epsilon$ , для каждого терминала  $b$  из  $FOLLOW(A)$  добавим  $A \rightarrow \alpha$  в ячейку  $M[A, b]$ . Если  $\epsilon$  входит в  $FIRST(\alpha)$ , а  $\$$  – в  $FOLLOW(A)$ , добавим  $A \rightarrow \alpha$  в ячейку  $M[A, \$]$
4. Сделать каждую неопределённую ячейку таблицы  $M$  указывающей на ошибку

Данный алгоритм может быть применён к любой грамматике  $G$  для построения таблицы разбора  $M$ . Однако для некоторых грамматик  $M$  в одной ячейке таблицы может оказаться несколько записей. Например, если  $G$  – неоднозначная или леворекурсивная грамматика. Как поступить в данном случае? Один выход состоит в преобразовании грамматики, устраняющем левую рекурсию, и левой факторизации, в надежде получить грамматику, в таблице анализа которой отсутствуют ячейки с несколькими записями (пытливый студент отправляется к дополнительной литературе за разъяснениями по этому поводу). К сожалению, имеются грамматики, никакие изменения которых не приведут к желаемому результату. Грамматика в задании к данной лабораторной работе лишена подобной неоднозначности (однако настоятельно рекомендую проверить – это займёт не так много времени, как кажется, зато избавит от многочасовых (и, скорее всего, безрезультатных) поисков ошибки в вашем коде в том случае, если я ошибся)

### **Восстановление после ошибок в предиктивном анализе**

Синтаксический анализатор находит ошибку в тот момент, когда терминал на вершине стека не соответствует очередному входному символу или на вершине стека находится нетерминал  $A$ , очередной входной символ –  $a$ , а ячейка таблицы синтаксического анализа  $M[A, a]$  пуста.

Восстановление после ошибки в «режиме паники» основано на пропуске символов из входного потока до тех пор, пока не будет обнаружен токен из выбранного множества синхронизирующих токенов. Эффективность этого метода зависит от выбора синхронизирующего множества. Множества должны выбираться так, чтобы синтаксический анализатор быстро восстанавливался после часто встречающихся ошибок. Вот некоторые эвристические правила.

- В качестве первого приближения можно поместить в синхронизирующее множество для нетерминала  $A$  все символы из множества  $FOLLOW(A)$ .
- Множества  $FOLLOW(A)$  недостаточно. Например, если инструкция завершается точкой с запятой, то ключевое слово, с которого начинается инструкция, может не оказаться во множестве  $FOLLOW$  нетерминалов, порождающих выражения. Отсутствие точки с запятой после присвоения может, таким образом, привести к пропуску ключевого слова, с которого начинается новая инструкция. Зачастую в языке имеется иерархическая структура конструкций – например, выражения появляются в инструкциях, которые находятся в блоках и т.д. В таком случае к синхронизирующему множеству конструкции нижнего уровня можно добавить символы, с которых начинаются конструкции более высокого уровня. Например, можно добавить

ключевые слова, с которых начинаются инструкции, в синхронизирующие множества нетерминалов, порождающих выражения.

- Если добавить символы из  $FIRST(A)$  в синхронизирующее множество для нетерминала  $A$ , станет возможным продолжение анализа в соответствии с  $A$ , когда во входном потоке появится символ из множества  $FIRST(A)$ .
- Если нетерминал может порождать пустую строку, то в качестве продукции по умолчанию может использоваться продукция, порождающая  $\epsilon$ . Это может отсрочить обнаружение ошибки, зато не может вызвать её пропуск и сокращает число нетерминалов, которые должны быть рассмотрены в процессе восстановления после ошибки.
- Если терминал на вершине стека не может быть сопоставлен с входным символом, то простейший метод состоит в снятии терминала со стека, генерации сообщения о вставке терминала в программу и продолжении синтаксического анализа. По сути, при этом подходе синхронизирующее множество токена состоит из всех остальных токенов.

### Пример 3

Использование символов из множеств FOLLOW и FIRST в качестве синхронизирующих достаточно неплохо работает при разборе грамматики из примера 1. Таблица синтаксического анализа для этой грамматики показана ниже. “Synch” указывает синхронизирующие токены, полученные из множества FOLLOW соответствующего терминала. Множества FOLLOW для нетерминалов получены из примера 2.

| Нетерминал | Входной символ     |                          |                     |                     |                          |                          |
|------------|--------------------|--------------------------|---------------------|---------------------|--------------------------|--------------------------|
|            | id                 | +                        | *                   | (                   | )                        | \$                       |
| E          | $E \rightarrow TA$ |                          |                     | $E \rightarrow TA$  | Synch                    | Synch                    |
| A          |                    | $A \rightarrow +TA$      |                     |                     | $A \rightarrow \epsilon$ | $A \rightarrow \epsilon$ |
| T          | $T \rightarrow FB$ | Synch                    |                     | $T \rightarrow FB$  | Synch                    | Synch                    |
| B          |                    | $B \rightarrow \epsilon$ | $B \rightarrow *FB$ |                     | $B \rightarrow \epsilon$ | $B \rightarrow \epsilon$ |
| F          | $F \rightarrow id$ | Synch                    | Synch               | $F \rightarrow (E)$ | Synch                    | Synch                    |

Таблица используется следующим образом. Если синтаксический анализатор просматривает ячейку  $M[A,a]$  и обнаруживает, что она пуста, то входной символ  $a$  пропускается. Если в этой ячейке находится запись Synch, то нетерминал снимается с вершины стека в попытке продолжить синтаксический анализ. Если токен на вершине стека не соответствует входному символу, то он снимается со стека.

**Пример 4.** В случае ошибочного ввода  $)id*+id$  синтаксический анализатор и механизм восстановления после ошибок поведут себя так:

| Стек   | Вход      | Примечание               |
|--------|-----------|--------------------------|
| \$E    | )id*+id\$ | Ошибка, пропускаем «)»   |
| \$E    | id*+id\$  | id $\in FIRST(E)$        |
| \$AT   | id*+id\$  |                          |
| \$ABF  | id*+id\$  |                          |
| \$ABid | id*+id\$  |                          |
| \$AB   | *+id\$    |                          |
| \$ABF* | *+id\$    |                          |
| \$ABF  | +id\$     | Ошибка, $M[F,+] = Synch$ |



|        |       |                      |
|--------|-------|----------------------|
| \$AB   | +id\$ | F снимается со стека |
| \$A    | +id\$ |                      |
| \$AT+  | +id\$ |                      |
| \$AT   | id\$  |                      |
| \$ABF  | id\$  |                      |
| \$ABid | id\$  |                      |
| \$AB   | \$    |                      |
| \$A    | \$    |                      |
| \$     | \$    |                      |

Приведённое обсуждение метода восстановления после ошибки в «режиме паники» не касалось важного вопроса сообщений об ошибках. Вообще говоря, разработчик компилятора должен обеспечить вывод информативных сообщений об обнаруженных ошибках.

*Восстановление на уровне фразы.* Реализуется заполнением пустых ячеек в таблице предиктивного синтаксического анализа указателями на подпрограммы обработки ошибок. Эти подпрограммы могут изменять, вставлять или удалять символы входного потока и выводить соответствующие сообщения об ошибках. Кроме того, они могут снимать элементы со стека. При таком способе восстановления возникает вопрос, можно ли разрешить изменение символов в стеке или только размещение в стеке новых символов, поскольку после этого шаги, выполняемые синтаксическим анализатором, могут не соответствовать порождению ни одного слова языка вообще. В любом случае нужно гарантировать, что программа не заикнется. Надёжная защита от заикливания – проверка того, что каждое действие по восстановлению после ошибки в конечном счёте приводит к обработке очередного входного символа (или к снятию элементов со стека, если достигнут конец входного потока).

### Задание на лабораторную работу

Написать синтаксический анализатор, обнаруживающий наибольшее число ошибок, для приведённой ниже грамматики (данная грамматика является упрощённым вариантом грамматики языка C):

```

<program>: <type> 'main' '(' ')' '{' <statement> '}'
<type>: 'int'
      | 'bool'
      | 'void'
<statement>:
      | <declaration> ';'
      | '{' <statement> '}'
      | <for> <statement>
      | <if> <statement>
      | <return>
<declaration>: <type> <identifier> <assign>
<identifier>: <character><id_end>
<character>: 'a' | 'b' | 'c' | 'd' | 'e' | 'f' | 'g' | 'h' | 'i' | 'j' | 'k' | 'l' | 'm' | 'n' | 'o' | 'p' | 'q' |
            'r' | 's' | 't' | 'u' | 'v' | 'w' | 'x' | 'y' | 'z' | 'A' | 'B' | 'C' | 'D' | 'E' | 'F' | 'G' |
            'H' | 'I' | 'J' | 'K' | 'L' | 'M' | 'N' | 'O' | 'P' | 'Q' | 'R' | 'S' | 'T' | 'U' | 'V' |
            'W' | 'X' | 'Y' | 'Z' | '_'

```

```

<id_end>:
    | <character><id_end>
<assign>:
    | '=' <assign_end>
<assign_end>: <identifier>
               | <number>
<number>: <digit><number_end>
<digit>: '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
<number_end>:
    | <digit><number_end>
<for>: 'for' '(' <declaration> ';' <bool_expression> ';' ')'
<bool_expression>: <identifier> <relop> <identifier>
                   | <number> <relop> <identifier>
<relop>: '<' | '>' | '==' | '!='
<if>: 'if' '(' <bool_expression> ')'
<return>: 'return' <number> ';'

```

Данная грамматика записана с помощью вариации РБНФ. Нетерминалы заключены в <>, терминал (или последовательность терминалов) – в ‘’. Символ | обозначает альтернативу (т.е. «или»). Запись <N1><N2> означает, конкатенацию нетерминалов, а запись <N1> <N2> – последовательную запись нетерминалов, разделённых пробельными символами (пробельные символы аналогичны таковым в грамматике языка C). Запись вида <N1>: | <N2> означает, что нетерминал N1 можно заменить либо нетерминалом N2, либо пустой строкой. Стартовым нетерминалом грамматики является <program>. Назовём язык, описываемый данной граматикой, C-light.

Входные данные: файл с программой на языке C-light

Выходные данные:

1. **Задание минимум:** количество обнаруженных ошибок и сообщения о месте их возникновения
2. **Задание максимум:** количество обнаруженных ошибок, сообщения о месте их возникновения и о их типе, файл с таблицей, аналогичной таблице из примера 4.

Восстановление после ошибок реализовать в «режиме паники»

В качестве **дополнения** может быть реализован механизм восстановления после ошибок в режиме фразы, а также сгенерирован файл с таблицей предиктивного анализа.

## Литература

При подготовке данной лабораторной работы использовалась книга Альфред Ахо, Рави Сети, Джеффри Ульман – Компиляторы. Принципы, технологии, инструменты – Вильямс – Москва Санкт-Петербург Киев 2003